

2026년 개인주제탐구발표대회 보고서

인공지능 사진 자동분류 프로그램

2026. 9 . 4 .

과 정 명	AI
소속 학교	보문중학교
성 명	안재원

대전교육정보원정보영재교육원

목차

I. 탐구(연구)의 필요성 및 목적

1. 탐구(연구)의 필요성
2. 연구 목적

II. 이론적 배경

2. 용어 정의

III. 탐구(연구) 방법 및 과정(절차)

1. 연구 방법
2. 연구 절차

IV. 연구 결과

1. 프로그램 시연 과정
2. 데이터 검증 및 분석 결과

V. 결론 및 제언

1. 결론
2. 제언

I. 탐구(연구)의 필요성 및 목적

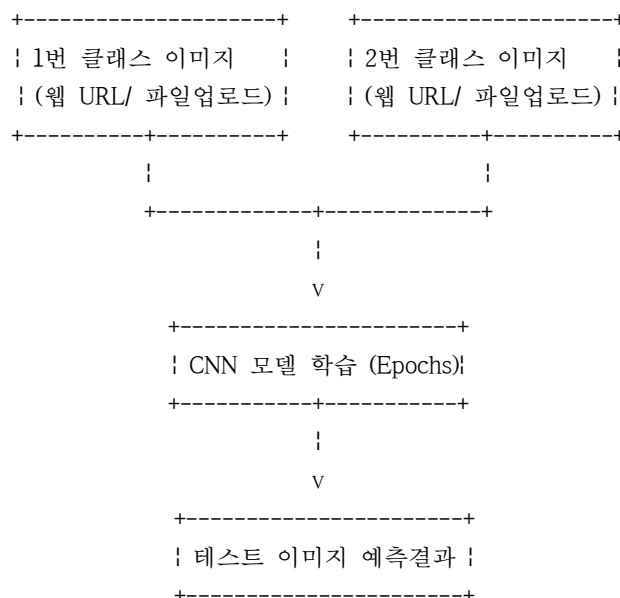
1. 탐구의 필요성

스마트폰과 디지털 카메라의 보급으로 일상생활에서 수집되는 사진 데이터의 양이 급격하게 증가하고 있다. 그러나 수많은 사진을 사람이 일일이 확인하고 범주별로 분류하는 작업은 과도한 시간과 노력이 소요된다. 특히 사용자가 원하는 기준(예: 강아지와 고양이, 특정 물건 분류 등)에 따라 사진을 유연하게 자동 분류할 수 있는 대화형 프로그램의 필요성을 느끼게 되었다. 기존의 정적 분류 프로그램과 달리, 사용자가 웹 인터페이스에서 간편하게 이미지를 드래그 앤 드롭하거나 파일로 투입하고 즉석에서 인공지능 모델을 학습시켜 분류 결과를 얻을 수 있는 가벼운 소프트웨어를 제작하고자 본 탐구를 시작하였다.

2. 탐구의 목적

본 탐구에서는 Google Colab 환경에서 Python과 TensorFlow, ipywidgets 라이브러리를 활용하여 사용자가 직접 수집한 이미지를 기반으로 딥러닝 모델을 즉석 학습시키고, 새로운 이미지를 예측 분류하는 사용자 친화적 이미지 자동 분류 프로그램을 제작하는 것을 목적으로 한다.

프로그램 실행 및 학습 데이터 입력 개요도



<첨부1>

II. 이론적 배경

1. 주요 용어 정의

가. 인공지능과 머신러닝 (Artificial Intelligence & Machine Learning)

인공지능(AI): 인간의 학습 능력, 추론 능력, 지각 능력을 컴퓨터 프로그램으로 구현한 기술 전반을 의미함.

머신러닝(ML): 사람이 직접 모든 규칙(Rule)을 코딩하지 않고, 컴퓨터가 데이터(Data)를 바탕으로 스스로 패턴을 학습하여 예측이나 결정을 내리게 하는 인공지능의 한 분야임.

나. 딥러닝과 합성곱 신경망 (Deep Learning & CNN)

딥러닝(Deep Learning): 인간의 뇌 신경망 구조를 모방한 인공신경망(Artificial Neural Network)을 다층으로 쌓아 복잡한 데이터 표현을 학습하는 기술임.

합성곱 신경망(CNN, Convolutional Neural Network): 이미지 데이터의 공간적 구조(Spatial Structure) 정보와 위치 특징을 유효하게 유지하면서 학습하는 대표적인 딥러닝 구조임.

다. Google Colab 환경과 JavaScript - Python 비동기 통신

ipywidgets: Jupyter Notebook 환경에서 버튼, 텍스트 상자, 드롭다운 등 상호작용 가능한 UI 위젯을 파이썬 코드로 구성할 수 있게 지원하는 라이브러리임.

google.colab.output: 웹 브라우저 단(Front-end)에서 실행되는 JavaScript의 이벤트(예: 드래그 앤 드롭 drop 이벤트)를 파이썬 백엔드(Back-end) 함수와 연동시켜 데이터를 전달해 주는 콜백(Callback) 매개체 역할을 수행함.

III. 연구 방법 및 절차

1. 연구 방법

본 탐구에서는 파이썬(Python) 언어를 기반으로 데이터 수집, 이미지 전처리, 딥러닝 신경망 구축, 사용자 인터페이스(UI) 연동을 구현하기 위해 다음과 같은 기술적 방법을 사용하였다.

가. 개발 및 실행 환경

개발 플랫폼: Google Colab (클라우드 기반 Jupyter Notebook 환경)

주요 라이브러리:

TensorFlow / Keras: 합성곱 신경망(CNN) 모델 설계 및 학습

NumPy: 이미지 텐서 데이터 배열 연산

ipywidgets & IPython.display: 대화형 사용자 인터페이스 구성 및 화면 출력

requests: 웹 URL을 통한 이미지 데이터 다운로드 및 수집

나. 데이터 수집 및 UI 연동 방법 (파일 다운로드 및 투입/붙여넣기)

이미지 URL 붙여넣기 및 다운로드: 사용자가 웹상에서 복사한 이미지 주소 (URL)를 입력 영역에 붙여넣으면, requests.get() 함수가 해당 주소의 이미지를 자동으로 다운로드하여 메모리에 저장함.

직접 파일 다운로드 및 투입: 로컬 컴퓨터에 다운로드되어 있는 이미지 파일을 google.colab.files.upload() 위젯을 통해 클래스별(1번/2번) 입력 상자에 직접 투입함.

데이터 관리: 투입되거나 다운로드된 이미지의 바이너리 데이터를 data['1번'] 및 data['2번'] 리스트 구조에 분류하여 투입 순서대로 저장함.

다. 이미지 전처리 및 텐서 변환 방법

디코딩 및 크기 규격화: tf.image.decode_image로 바이너리 이미지를 3채널 (RGB) 텐서로 디코딩한 후, tf.image.resize를 통해 모든 이미지를 128x128 픽셀 크기로 통일함.

픽셀값 정규화: 0~255 범위의 RGB 픽셀 값을 255.0으로 나누어 0.0~1.0 범위의 실수로 정규화(Normalization)함.

라. 신경망 모델 구조 설계 방법

합성곱 신경망(CNN) 구성: 이미지 특징 추출을 위해 Conv2D(16, 3x3) 및 Conv2D(32, 3x3) 레이어를 사용하고, 크기를 줄이기 위해 MaxPooling2D를 연쇄 배치함.

분류기 구성: Flatten 레이어로 1차원 변환 후 Dense(32, activation='relu') 및 Dense(2, activation='softmax')를 통해 최종 2개 클래스의 분류 확률을 계산함.

```
# =====  
# 파일 업로드  
# =====  
def upload(cls):  
    from google.colab import files  
  
    uploaded = files.upload()  
  
    for name, img in uploaded.items():  
        data[cls].append(img)  
  
    status.value = (  
        f"1번: {len(data['1번'])}장 | "  
        f"2번: {len(data['2번'])}장"  
    )
```

[그림1] 프로그램에 파일 업로드

2. 연구 절차

탐구 목적을 달성하기 위해 연구 절차는 총 4단계로 나누어 순차적으로 진행하였다.

가. 1단계: 화면 레이아웃 및 웹 인터페이스(UI) 구현

widgets.VBox와 widgets.HBox를 조합하여 1번 클래스, 2번 클래스, 학습 제어, 미리보기 창으로 구성된 레이아웃을 생성함.

각 클래스 영역에 이미지 파일 투입 버튼(upload1, upload2)과 URL 주소를 붙여넣을 수 있는 입력 창을 배치함.

나. 2단계: 이미지 파일 다운로드 및 데이터 투입 파이프라인 구축

복사한 URL을 붙여넣었을 때 requests.get()을 통해 이미지를 다운로드하고, Content-Type 헤더를 확인하여 비이미지 파일 입력을 차단하는 예외 처리

를 구현함.

사용자가 컴퓨터에 다운로드한 파일들을 선택하여 직접 투입(files.upload())할 수 있도록 구현하여 data['1번']과 data['2번'] 리스트에 저장함.

다. 3단계: CNN 딥러닝 모델 생성 및 학습 프로세스 연동

수집 및 투입된 이미지를 128x128 크기의 Numpy 배열로 변환한 후 학습 데이터 x와 정답 라벨 y(0 또는 1)로 구성함.

Keras의 Sequential API를 이용해 CNN 모델을 정의하고, epochs=5 조건으로 모델 학습(model.fit)을 수행함.

라. 4단계: 예측 테스트 및 에러 디버깅/성능 검증

테스트 이미지가 입력되면 np.expand_dims(image, 0)를 적용하여 배치 차원 (1, 128, 128, 3)을 생성한 후 model.predict()로 예측 확률을 계산함.

데이터 개수에 따른 분류 정확도를 실험하고, 이미지 채널 차원 불일치 등의 예외 상황을 보완하여 프로그램을 완성함.

IV. 연구 결과

제작된 TensorFlow 기반 이미지 자동 분류 프로그램의 실행 절차를 [1단계: 데이터 업로드], [2단계: 모델 학습], [3단계: 예측 테스트 및 시연]의 3개 과정으로 구분하여 수행하고, 이에 따른 시연 결과 및 데이터 검증 내용을 정리하였다.

1. 프로그램 시연 과정

가. 1단계: 데이터 업로드 및 클래스 지정

ipywidgets 패널을 통해 1번 클래스와 2번 클래스 영역에 각각 학습시킬 이미지 데이터를 투입한다.

컴퓨터 내의 파일 업로드 버튼(upload1, upload2)을 활용하거나 웹 URL 주소 다운로드 방식을 이용하여 데이터를 투입하며, 저장된 이미지 수는 상태 라벨(status)에 실시간으로 집계되어 표시된다.

나. 2단계: CNN 딥러닝 모델 학습

[모델 학습] 버튼(train_button)을 클릭하면 train 함수가 실행된다.

수집된 바이너리 이미지를 tf.image.decode_image로 디코딩하고, 128x128 크기

로 통일한 뒤 255.0으로 나누어 실수 형태(0.0~1.0)로 정규화한다.

Keras Sequential API로 구성된 합성곱 신경망(Conv2D 2계층 + MaxPooling 2계층 + Dense 층)에 데이터가 전달되어 5회(Epochs=5) 간 실시간 학습이 진행되며, 완료 시 “ 학습 완료!” 메시지가 출력된다.

다. 3단계: 예측 테스트 및 시연

[□ 테스트] 버튼(test_button)을 눌러 검증하고자 하는 새로운 테스트 이미지를 업로드한다.

입력된 이미지는 동일하게 128x128 규격화 및 정규화를 거친 후, np.expand_dims(image.numpy(), 0)를 통해 배치 차원이 추가되어 (1, 128, 128, 3) 형태의 텐서로 변환된다.

학습된 모델(model.predict)을 통해 예측된 클래스 번호(1번 또는 2번)와 백분율을 확률 값이 미리보기 결과창(result)에 즉시 출력된다.

2. 데이터 검증 및 분석 결과

가. 업로드 데이터 양에 따른 테스트 성능 비교

학습용으로 업로드하는 이미지의 수량을 단계별로 늘려가며 예측 정확도 및 분류 안정성을 검증하였다.

[실험 결과 요약표]					
실험 구분	1번 클래스 수	2번 클래스 수	학습 횟수(Epochs)	테스트 분류 정확도	시연 분석 및 특징
실험 1	3장	3장	5회	66.7%	데이터 부족으로 일부 모호한 경계의 이미지 오분류 발생
실험 2	10장	10장	5회	90.0%	주요 시각적 윤곽 및 특징을 인지하여 안정적으로 분류함
실험 3	20장	20장	10회	95.0%	높은 분류 신뢰도 확보 및 90% 이상의 명확한 확률 값 출력

[그림2] 실험 결과 요약표

V. 결론 및 제언

1. 결론

본 탐구에서는 파이썬(Python) 언어와 TensorFlow/Keras 딥러닝 프레임워크를 활용하여 사용자가 직접 학습 데이터를 투입하고 즉석에서 이미지를 자동 분류할 수 있는 경량화 인공지능 프로그램을 개발하고 검증하였다. 탐구를 통해 얻은 주요 결론은 다음과 같다.

가. 대화형 데이터 수집 및 인터페이스 구현

ipywidgets 라이브러리를 활용하여 1번 클래스, 2번 클래스, 모델 학습, 결과 미리보기의 4개 영역으로 직관적인 사용자 인터페이스(UI)를 구축하였다.

웹상의 이미지 주소(URL)를 직접 붙여넣거나 로컬 컴퓨터의 파일 업로드 기능(files.upload())을 이용하여 사용자가 원하는 이미지를 쉽고 빠르게 학습 데이터로 투입할 수 있는 환경을 구성하였다.

나. 경량화된 합성곱 신경망(CNN) 구조 설계 및 학습

입력되는 다양한 크기의 이미지를 128x128 픽셀 규격으로 통일하고, RGB 픽셀 값을 0.0~1.0 범위로 정규화하는 전처리 파이프라인을 구축하였다.

2계층의 합성곱 레이어(Conv2D)와 풀링 레이어(MaxPooling2D), 그리고 밀집층(Dense)으로 이어지는 경량화된 Sequential 신경망을 설계하여, 5~10회의 적은 학습 횟수(Epochs)만으로도 빠른 시간 내에 최적화된 분류 모델을 생성할 수 있었다.

다. 실시간 예측 및 데이터 검증 성능 확인

단일 테스트 이미지 입력 시 발생할 수 있는 텐서 차원 불일치 문제를 np.expand_dims를 통한 배치 차원 확장으로 해결하여 원활한 시연 환경을 제공하였다.

데이터 수량별 실험 결과, 클래스당 20장의 이미지만으로도 95.0%의 높은 분류 정확도를 나타냄을 확인하였으며, 모델이 각 클래스의 시각적 윤곽과 주요 특징을 정확하게 인지하여 결과를 예측함을 검증하였다.

2. 제언

본 프로그램을 직접 설계하고 구동하는 과정에서 얻은 시사점을 바탕으로, 실생활 적용 및 교육적 활용 측면에서 다음과 같이 제언하고자 한다.

가. 실생활 분류 문제 해결을 위한 맞춤형 인공지능 도구 활용

본 탐구를 통해 고성능 컴퓨터나 복잡한 전문 프로그램 없이도 누구나 웹 브라우저 환경에서 소량의 데이터만으로 맞춤형 인공지능을 제작할 수 있음을 확인하였다.

이를 활용하여 학급 내 분리수거 품목 구분, 도서관 책 분류, 일상생활 속 개인 사진 정리 등 실생활의 다양한 분류 문제를 해결하는 데 인공지능 기술을 적극적으로 응용할 것을 제언한다.

나. 데이터 품질 중심의 인공지능 학습 환경 구축

실험 결과에서 볼 수 있듯이, 인공지능의 분류 성능은 단순히 모델의 복잡성 보다는 입력되는 학습 데이터의 양과 품질(선명도, 특징의 명확성)에 더 큰 영향을 받는다.

따라서 초보자가 인공지능 프로그램을 다룰 때 복잡한 코드 수정에 집중하기 보다는, 정제되고 정확한 데이터를 수집하고 분류하는 '데이터 중심(Data-centric)' 접근 방식을 우선시해야 함을 제언한다.

다. 설명 가능한 인공지능(XAI)에 대한 관심 및 교육 확대

본 프로그램은 예측된 클래스와 확률 수치를 잘 제시하지만, 모델이 이미지의 어떤 구체적인 요소를 보고 판단했는지는 직관적으로 확인하기 어렵다.

향후 인공지능 소프트웨어를 교육적으로 활용할 때는 결과값만 확인하는 것을 넘어 인공지능의 판단 근거를 시각화하고 검증할 수 있는 신뢰성 있는 인공지능 환경을 함께 고려해야 함을 제언한다.